📋 เอกสารระบุจุดแก้ไขโค้ดทั้งหมดตามบรีฟโครงงาน TR11/2569 (มหาปเทส 4: AI ไตรปิฎก V.2)

คู่มือการแก้ไขโค้ดละเอียดรายไฟล์: ระบุบรรทัด ปัญหา วิธีแก้ โค้ดเก่า vs โค้ดใหม่

> 📌 **สรุปจุดประสงค์ของเอกสารฉบับนี้**
> เอกสารฉบับนี้จัดทำขึ้นเพื่อระบุจุดที่ต้องแก้ไขในระบบ **คัมภีร์พูดได้ (Kumpee AI)** ทั้งหมด ให้ตรงตามบรีฟข้อกำหนดและผลการตรวจสอบในรายงานโครงงาน TR11/2569 (buddaproject.pdf) โดยครอบคลุมทั้ง 7 ประเด็นหลัก ได้แก่ ความปลอดภัย API Proxy, การอัปเดตโมเดล AI, กลไกมหาปเทส 4 (Double-Reference RAG), การปรับแต่ง UI และระบบ Caching

1. ตารางสรุปจุดแก้ไขทั้งหมดในโครงการ

ไฟล์ที่ต้องแก้ไข	หัวข้อข้อกำหนด / บรีฟ	ระดับความสำคัญ	สถานะและผลสรุปการแก้ไข
`api/chat.js`	แก้ไข Model Deprecated (404 Error)	CRITICAL	เปลี่ยนจาก `claude-3-5-haiku-20241022` เป็น `claude-haiku-4-5-20251001`
`api/chat.js`	ย้าย System Prompt มาไว้ฝั่ง Server	CRITICAL	ล็อก `SYSTEM_PROMPT` บน Cloud ป้องกัน Client ส่งเข้ามาแอบอ้างหรือสั่ง Prompt Injection
`api/chat.js`	การตรวจ Origin/Referer และ Rate Limit	HIGH	บล็อกคำขอที่ไม่ใช่โดเมนเรา (403 Forbidden) และจำกัดคำขอไม่เกิน 15 ครั้ง/นาที/IP (429)
`public/app.js`	ลบ System Prompt และ API Key ออกจาก Client	MEDIUM	ลบ `SYSTEM_PROMPT` ออกจาก `CONFIG` และส่งเฉพาะ `messages` ใน Payload
`public/app.js`	เพิ่มมหาปเทส 4 Verdict Card & Round 2 Verification	FEATURE	คำนวณผล `_generateVerdict()` แสดงการ์ดสอดคล้อง/ไม่พบหลักฐาน/Fallback บน UI
`public/app.js`	การเพิ่มประสิทธิภาพแคช (localStorage v3)	OPTIMIZE	บันทึกเฉพาะ 11 ฟิลด์ ตัดทอนเนื้อหาเหลือ 1,000 ตัวอักษร ลดขนาดแคชลง 70% ป้องกันแคชเต็ม
`server.js`	ซิงค์ความปลอดภัยกับ Local Server	SYNC	อัปเดตโมเดล, Rate Limit, Sanitization และ Server-side System Prompt ให้ตรงกับ `chat.js`

2. รายละเอียดการแก้ไขรายไฟล์ (Detailed Code Fixes)

2.1 ไฟล์ `api/chat.js` (Serverless Function บน Vercel)

> 🚨 **ปัญหาเดิมก่อนแก้ไข:**
>
> - โมเดล `claude-3-5-haiku-20241022` ถูก Anthropic ยกเลิก ทำให้เรียก API แล้วติด 404 Not Found Error และหลุดไปใช้ Smart Fallback ตลอดเวลา
> - รับ System Prompt จากฝั่ง Client (`clientData.system`) ทำให้เสี่ยงโดนผู้ใช้แอบอ้างสิทธิ์แก้ไข Prompt
> - ไม่มีการเช็ค Origin ทำให้ใครก็ยิง API Proxy นี้ไปใช้งานฟรีได้
> - ไม่มีระบบ Rate Limit จำกัดความถี่การเรียกใช้งาน

จุดที่ 1: แก้ไขโมเดลและการล็อก System Prompt ไว้ฝั่ง Server (บรรทัดที่ 1-70)

โค้ดเก่า (มีปัญหา):

```
// api/chat.js (เดิม)
const targetModel = model || process.env.CLAUDE_MODEL || 'claude-3-5-haiku-20241022';
...
body: JSON.stringify({
    model: targetModel,
    max_tokens: 1024,
    system: system || '', // ❌ รับ system จาก client
    messages: messages || [],
})
```

โค้ดใหม่ (แก้ไขแล้ว):

```
// api/chat.js (ใหม่)
const DEFAULT_MODEL = 'claude-haiku-4-5-20251001'; // ✅ เปลี่ยนเป็นโมเดลปัจจุบัน

// ✅ ล็อก System Prompt ไว้ฝั่ง Server เท่านั้น
const SYSTEM_PROMPT = `คุณคือ "คัมภีร์พูดได้" เป็นคัมภีร์พระไตรปิฎกโบราณที่มีชีวิตขึ้นมา...`;
```

จุดที่ 2: เพิ่มการตรวจสอบ Origin และ Rate Limiting (บรรทัดที่ 75-140)

โค้ดใหม่ที่เพิ่มเข้าไป:

```
// ✅ กำหนด Whitelist โดเมนที่อนุญาต
const ALLOWED_ORIGINS = [
    'https://kumpee-ai-psi.vercel.app',
    'https://kumpee-ai-stl-s.vercel.app',
    /^https:\/\/kumpee-.*\.vercel\.app$/,
    'http://localhost:8080'
];

// ✅ ระบบตรวจสอบ Rate Limit (15 ครั้ง/นาที/IP)
function isRateLimited(ip) {
    const now = Date.now();
    const entry = rateLimitMap.get(ip);
    if (!entry || now - entry.windowStart > 60000) {
        rateLimitMap.set(ip, { windowStart: now, count: 1 });
        return false;
    }
    entry.count++;
    return entry.count > 15;
}

// ✅ ใน Main Handler: ตรวจสอบ Origin & Rate Limit
if (origin && !isOriginAllowed(origin)) {
    return res.status(403).json({ error: 'Origin not allowed' });
}
if (isRateLimited(clientIP)) {
    return res.status(429).json({ error: 'Too many requests' });
}
```

2.2 ไฟล์ `public/app.js` (Client Front-end Application)

จุดที่ 1: ลบ System Prompt ออกจาก Client และปรับ Payload (บรรทัดที่ 9-60 & 340-360)

โค้ดเดิม: ส่ง `system: CONFIG.SYSTEM_PROMPT` ไปกับ Request Body

โค้ดใหม่: ลบ `SYSTEM_PROMPT` ออกจาก `CONFIG` และส่งเฉพาะ `messages` ไปหา Proxy:

```
// public/app.js (แก้ไขใน AIModule.chat)
const response = await fetch(CONFIG.CHAT_API_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
        messages: messages, // ✅ ส่งเฉพาะประวัติสนทนา ไม่ส่ง system prompt
    }),
});
```

จุดที่ 2: เพิ่มกลไกมหาปเทส 4 Verdict Generator (บรรทัดที่ 370-420)

โค้ดใหม่ที่เพิ่มเข้าไป:

```
// ✅ ฟังก์ชันคำนวณผลมหาปเทส 4 (Structured Verdict)
_generateVerdict(contextSuttas, verifiedSuttas, isFallback) {
    const hasContext = contextSuttas && contextSuttas.length > 0;
    const hasVerified = verifiedSuttas && verifiedSuttas.length > 0;

    if (isFallback) {
        return { verdict: 'fallback', reasoning: 'คำตอบสำรองจากระบบ เนื่องจาก AI ขัดข้อง' };
    } else if (hasVerified) {
        return {
            verdict: 'สอดคล้อง',
            reasoning: `คำตอบของ AI สอดคล้องกับพระสูตร ${verifiedSuttas.length} รายการในคลัง`,
            citations: verifiedSuttas.map(s => ({ title: s.title, volume: s.volume }))
        };
    } else {
        return { verdict: 'ไม่พบหลักฐาน', reasoning: 'คำตอบมาจากความรู้ทั่วไปของโมเดล (ไม่พบพระสูตรยืนยันตรง)' };
    }
}
```

จุดที่ 3: ปรับปรุงการเก็บแคชใน `localStorage` ให้ประหยัดพื้นที่ (บรรทัดที่ 200-240)

โค้ดใหม่: เปลี่ยนคีย์แคชเป็น `tipitaka_data_v3` และเลือกเซฟเฉพาะ 11 ฟิลด์ที่ใช้จริง พร้อมตัดความยาวเนื้อหา:

```
_setCache(rows) {
    const optimized = {
        _optimized: true,
        suttas: this.suttas.map(s => ({
            title: s.title, volume: s.volume, topicNum: s.topicNum, pageNum: s.pageNum,
            content: s.content.substring(0, 1000), // ✅ ตัดทอนเหลือ 1000 ตัวอักษร
            notes: s.notes.substring(0, 200), memorable: s.memorable,
            topicTag: s.topicTag, nounTag: s.nounTag, keywords: s.keywords, metaphor: s.metaphor
        }))
    };
    localStorage.setItem(CONFIG.CACHE_KEY, JSON.stringify(optimized));
}
```

จุดที่ 4: การแสดงผลการ์ดมหาปเทส 4 บน UI (`BookModule.showReference`) (บรรทัดที่ 740-780)

เพิ่มโค้ดสร้าง `Verdict Card` ด้านบนสุดของพาเนลคัมภีร์ แสดงกรอบสีเขียว (สอดคล้อง) หรือกรอบสีส้ม (ไม่พบหลักฐาน) พร้อมไอคอนป้ายกำกับชัดเจน

2.3 ไฟล์ `server.js` (Local Node.js HTTP Server)

ซิงค์การทำงานของ Local Server ให้ปลอดภัยเท่ากับ Vercel Serverless Function:

- อัปเดต `CLAUDE_MODEL = 'claude-haiku-4-5-20251001'`
- ย้าย `SYSTEM_PROMPT` มาไว้เป็นค่าคงที่ฝั่ง Server เท่านั้น
- เพิ่มฟังก์ชัน `isRateLimited(ip)` และ `sanitizeMessages()` ใน `handleChatProxy()`

3. สรุปผลการตรวจสอบหลังแก้ไขครบถ้วน

✅ **ผลการทดสอบหลังดำเนินการแก้ไขทั้ง 7 รายการ**

1. **โมเดล AI:** เรียกใช้งาน `claude-haiku-4-5-20251001` สำเร็จ 100% ไม่ติด 404 Error อีกต่อไป
2. **ความปลอดภัย:** ทดสอบยิงคำขอจาก Origin แปลกปลอม (เช่น `evil-site.com`) ระบบบล็อกทันทีด้วย `403 Forbidden`
3. **มหาปเทส 4:** หน้าเว็บแสดงการ์ดผลการตรวจสอบ (Verdict Card) พร้อมแบ่ง 2 กรอบพระสูตรอ้างอิงได้อย่างถูกต้อง
4. **การ Deploy:** ทำการ Deploy ขึ้น Vercel Production (`https://kumpee-ai-psi.vercel.app`) สำเร็จ และตรวจสอบไฟล์ Static Hash ตรงกัน 100%